

RotaVital

Sistema de logística de distribuição de hemocomponentes

Documento de Arquitetura da Aplicação

Diagrama de arquitetura · endpoints REST · ligações e protocolos · redes

INSTITUIÇÃO	CESAR School — Análise e Desenvolvimento de Sistemas
TURMA	ADS — 3º período
DATA	26/09/2026
INTEGRANTES	Victor José Paes e Silva (líder) Eduardo de Souza Cavalcanti Junior Felipe Franca Alves de Lima Helamã Leone de Lima Procídio João Pedro Arruda Guimarães Júlia Oliveira Veríssimo Lucas Pagueti Pereira Tiago Luiz Moreira de Vasconcelos
ARQUIVOS	Arquitetura_RotaVital_Entrega_RSD.pdf drawio_atualizado.png diagrama-arquitetura.drawio

2 Descrição do sistema

O **RotaVital** é um sistema web de logística para a distribuição de hemocomponentes (concentrado de hemácias, plasma e plaquetas) entre hemocentros, bancos de sangue e hospitais. Ele reúne em um só lugar a gestão de estoque, as requisições hospitalares, a checagem de compatibilidade ABO/Rh, a roteirização das entregas e o acompanhamento da temperatura da cadeia fria. O objetivo é garantir que a bolsa certa chegue ao paciente certo, no tempo certo, com rastreabilidade e controle de validade.

Quem usa

Usuário	O que faz no sistema
Profissional de hemocentro	Cadastra bolsas, acompanha estoque e validade, atende requisições alocando bolsas compatíveis.
Médico	Abre requisições de hemocomponentes para pacientes e acompanha o andamento.
Gestor hospitalar	Acompanha o estoque consolidado, a priorização das requisições e as entregas em trânsito.
Doador (portal de doações)	Consulta informações do portal de doações.

Principais funcionalidades

- Visão do estoque por banco de sangue, tipo ABO/Rh e hemocomponente.
- Requisições hospitalares com prioridade e alocação automática por **FEFO** (vence primeiro, sai primeiro) e compatibilidade.
- Rede de distribuição modelada como grafo (pontos e conexões) e cálculo de rota entre origem e destino, com mapa.
- Registro e histórico das leituras de temperatura de cada entrega (cadeia fria).

Estado atual do MVP e como ler o diagrama

Hoje o projeto roda em dois containers do docker-compose.yml: **frontend** (Nginx servindo a SPA React) e **backend** (Spring Boot). A SPA ainda consome dados simulados (frontend/src/data/*Mock.ts) e o backend guarda o estado em memória (HashMap); a conexão com o PostgreSQL/Supabase é testada no boot, mas nenhum dado é gravado ainda. Por isso o diagrama separa ligações **ativas** (pretas) de ligações **planejadas** (laranja). As redes, sub-redes e o HTTPS na borda representam o **ambiente de produção projetado**, como pede a atividade.

3 Diagrama de arquitetura

RotaVital — Diagrama de Arquitetura da Aplicação

Ambiente de produção projetado · fluxo da esquerda (usuário) para a direita (banco) · todo rótulo de seta segue o formato **PROTOCOLO / porta**

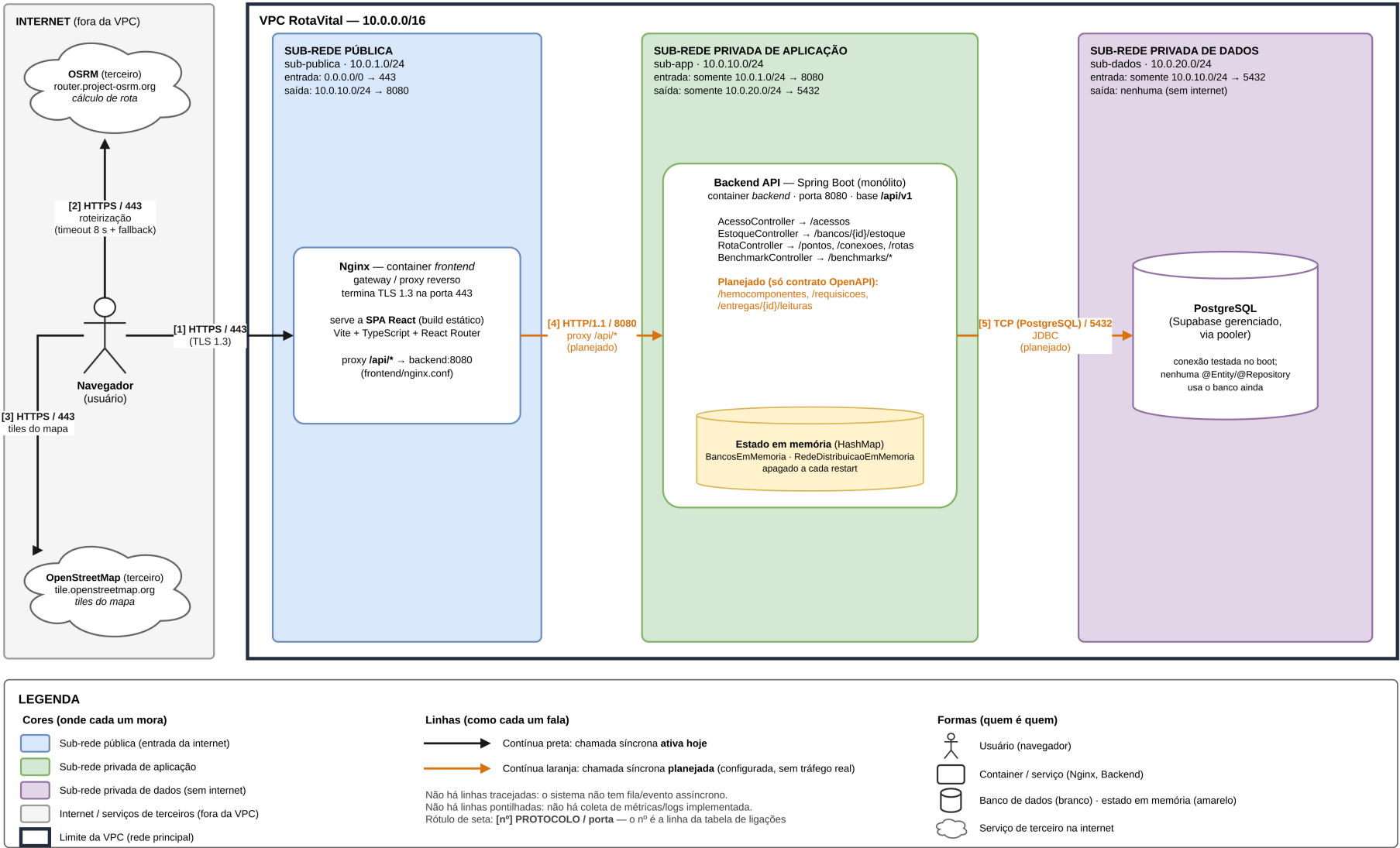


Figura 1 — Diagrama de arquitetura exportado do draw.io (PNG, zoom 300%). Arquivo-fonte: diagrama-arquitetura.drawio. Setas [1]-[5] = linhas da tabela da seção 5.

3.1 Inventário de componentes

Lista de tudo que executa código ou guarda dado no RotaVital (etapa 1 da atividade). Todo item daqui aparece no diagrama, e nada aparece no diagrama sem estar aqui.

Componente	O que é / o que faz	Onde mora	Situação
Navegador (SPA React)	Inicia todas as requisições; executa a SPA (Vite + TypeScript + React Router).	Internet	Ativo
Nginx (container frontend)	Serve o build estático da SPA e faz proxy reverso de /api/* para o backend. É o único ponto de entrada (gateway) e termina o TLS.	sub-publica	Ativo (proxy ainda sem tráfego real)
Backend API (container backend)	Monólito Spring Boot, porta 8080, base /api/v1. Controllers: Acesso, Estoque, Rota, Benchmark.	sub-app	Ativo
Estado em memória	HashMaps BancosEmMemoria e RedeDistribuicaoEmMemoria, dentro do processo do backend (sem rede). Apagados a cada restart.	sub-app (dentro do backend)	Ativo (volátil)
PostgreSQL (Supabase)	Banco relacional gerenciado, acessado via pooler. Ainda sem @Entity/@Repository.	sub-dados	Planejado (conexão testada no boot)
OSRM (terceiro)	API pública de roteirização (router.project-osrm.org), chamada pelo navegador com timeout de 8 s e fallback.	Internet	Ativo
OpenStreetMap (terceiro)	Servidor de tiles do mapa (tile.openstreetmap.org), chamado pelo navegador.	Internet	Ativo

O que não existe no projeto (e por isso não está no diagrama): fila ou mensageria, cache, balanceador de carga separado (o Nginx cumpre esse papel), servidor de e-mail e ferramenta de observabilidade (Prometheus, Grafana etc.).

4 Tabela de endpoints REST

Todos os endpoints são servidos pelo **Backend API** na base `/api/v1` e chegam ao navegador pelo Nginx em `https://<domínio>/api/v1/...` Como o MVP passa de 15 operações, elas foram agrupadas por recurso (4.1) e as principais foram detalhadas (4.2).

4.1 Resumo por recurso

Recurso	Caminho base	Métodos	Controller	Situação
Acessos	<code>/api/v1/aceessos</code>	GET, POST	AcessoController	Implementado (memória)
Estoque de um banco	<code>/api/v1/bancos/{id}/estoque</code>	GET	EstoqueController	Implementado (memória)
Rede de distribuição	<code>/api/v1/pontos</code> <code>/api/v1/conexoes</code>	GET	RotaController	Implementado (memória)
Rotas	<code>/api/v1/rotas</code>	POST	RotaController	Implementado (memória)
Benchmarks de rota	<code>/api/v1/benchmarks/*</code>	GET	BenchmarkController	Implementado
Hemocomponentes (bolsas)	<code>/api/v1/hemocomponentes</code>	GET, POST, PATCH, DELETE	—	Planejado (contrato OpenAPI)
Requisições e alocações	<code>/api/v1/requisicoes</code> <code>/api/v1/requisicoes/{id}/alocacoes</code>	GET, POST, PATCH	—	Planejado (contrato OpenAPI)
Leituras de temperatura	<code>/api/v1/entregas/{id}/leituras</code>	GET, POST	—	Planejado (contrato OpenAPI)

4.2 Endpoints detalhados

A autenticação ainda não existe no MVP. Nos endpoints planejados, a coluna indica o perfil previsto (token JWT): **401** = sem token ou token inválido; **403** = token válido, mas sem o perfil exigido.

Método	Caminho	Descrição	Autenticação	Resposta	Status
Implementados (dados em memória)					
GET	<code>/api/v1/bancos/{id}/estoque</code>	Estoque consolidado do banco de sangue por tipo ABO/Rh e hemocomponente	Não (MVP)	EstoqueBanco	200, 404
GET	<code>/api/v1/pontos</code>	Lista os pontos (hemocentros e hospitais) do grafo de distribuição	Não (MVP)	List<Ponto>	200
GET	<code>/api/v1/conexoes</code>	Lista as conexões (trechos) entre os pontos	Não (MVP)	List<Conexao>	200
POST	<code>/api/v1/rotas</code>	Calcula a rota entre origem e destino. Corpo: {origemId, destinoId}. Nada é persistido.	Não (MVP)	Rota	200, 400, 404
Planejados (contrato OpenAPI)					
GET	<code>/api/v1/hemocomponentes</code> <code>?abo=0&rh=NEG</code> <code>&status=DISPONIVEL</code> <code>&page=0&size=20</code>	Lista bolsas com filtros e paginação	JWT (HEMOCENTRO, GESTOR)	Page<Hemocomponente Resumo>	200, 401, 403
POST	<code>/api/v1/hemocomponentes</code>	Registra uma nova bolsa	JWT (HEMOCENTRO)	Hemocomponente	201, 400, 401, 403

Método	Caminho	Descrição	Autenticação	Resposta	Status
GET	/api/v1/hemocomponentes/{id}	Detalha uma bolsa	JWT (HEMOCENTRO, GESTOR)	Hemocomponente	200, 401, 404
PATCH	/api/v1/hemocomponentes/{id}	Atualiza o status da bolsa (ex.: DISPONIVEL → DESCARTADA)	JWT (HEMOCENTRO)	Hemocomponente	200, 400, 404, 409
DELETE	/api/v1/hemocomponentes/{id}	Remove um cadastro (409 se a bolsa já estiver alocada)	JWT (HEMOCENTRO)	—	204, 404, 409
POST	/api/v1/requisicoes	Cria uma requisição hospitalar	JWT (MEDICO)	Requisicao	201, 400, 401, 403
GET	/api/v1/requisicoes?status=PENDENTE&prioridade=URGENTE&page=0&size=20	Lista requisições com filtros e paginação	JWT (MEDICO, HEMOCENTRO, GESTOR)	Page<Requisicao Resumo>	200, 401
GET	/api/v1/requisicoes/{id}	Detalha uma requisição	JWT (dono ou HEMOCENTRO)	Requisicao	200, 403, 404
POST	/api/v1/requisicoes/{id}/alocacoes	Aloca uma bolsa compatível (ABO/Rh + FEFO) à requisição	JWT (HEMOCENTRO)	Alocacao	201, 404, 409
PATCH	/api/v1/requisicoes/{id}	Cancela a requisição. Corpo: {"status": "CANCELADA"}	JWT (dono ou GESTOR)	Requisicao	200, 404, 409
POST	/api/v1/entregas/{id}/leituras	Registra uma leitura de temperatura da entrega	JWT (HEMOCENTRO)	Leitura	201, 400, 404
GET	/api/v1/entregas/{id}/leituras	Histórico de temperatura da entrega	JWT (HEMOCENTRO, GESTOR)	List<Leitura>	200, 404

Padrões REST aplicados: substantivos no plural e nenhum verbo na URL; versão no caminho (/api/v1); sub-recursos por hierarquia (/bancos/{id}/estoque, /requisicoes/{id}/alocacoes, /entregas/{id}/leituras); filtros e paginação por query string; 201 para criação, 204 para exclusão, 404 para recurso inexistente e 409 para conflito de estado.

5 Tabela de ligações e protocolos

Cada linha corresponde a uma seta do diagrama, com o mesmo número [1]-[5].

#	Origem	Destino	Protocolo	Porta	Situação
1	Navegador	Nginx (frontend)	HTTPS (HTTP/1.1 ou HTTP/2 sobre TLS 1.3)	443	Ativa
2	Navegador	OSRM (terceiro)	HTTPS	443	Ativa
3	Navegador	OpenStreetMap tiles (terceiro)	HTTPS	443	Ativa
4	Nginx (frontend)	Backend API	HTTP/1.1 (proxy reverso de /api/*)	8080	Planejada — proxy configurado; a SPA ainda usa mocks
5	Backend API	PostgreSQL (Supabase)	Protocolo PostgreSQL sobre TCP (driver JDBC, com TLS)	5432	Planejada — só o teste de conexão no boot

- **Borda sempre em HTTPS:** tudo que vem da internet chega ao Nginx em HTTPS/443. O TLS termina no Nginx; dali para dentro, na rede privada, o tráfego segue em HTTP/1.1 na porta 8080.
- **Banco nunca exposto:** só o Backend API alcança a porta 5432; nenhuma seta sai da internet para o banco.
- **Sem comunicação assíncrona:** não há fila nem eventos no projeto, por isso não há linha tracejada.
- **Terceiros chamados pelo navegador:** OSRM e tiles são acessados direto do navegador do usuário, não pelo servidor. Assim nenhum componente da VPC precisa sair para a internet, e não é necessário NAT Gateway.
- **Ambiente local:** no docker-compose de desenvolvimento o Nginx atende em localhost:80. Em produção ele escuta apenas na 443 com certificado TLS (listen 443 ssl no nginx.conf).

6 Tabela de definição de redes

Rede / sub-rede	CIDR	Tipo	Componentes	Entrada permitida	Saída permitida
VPC RotaVital	10.0.0.0/16	Rede principal	Todas as sub-redes abaixo	—	—
sub-publica	10.0.1.0/24	Pública	Nginx (container frontend)	0.0.0.0/0 → TCP 443	10.0.10.0/24 → TCP 8080
sub-app	10.0.10.0/24	Privada de aplicação	Backend API (+ estado em memória)	10.0.1.0/24 → TCP 8080	10.0.20.0/24 → TCP 5432
sub-dados	10.0.20.0/24	Privada de dados	PostgreSQL (Supabase)	10.0.10.0/24 → TCP 5432	Nenhuma (sem internet)

6.1 Regras de acesso (firewall / security groups)

Regra	Origem	Destino	Porta	Ação	Motivo
R1	Internet (0.0.0.0/0)	sub-publica (Nginx)	TCP 443	Permitir	Acesso HTTPS dos usuários
R2	sub-publica	sub-app (Backend)	TCP 8080	Permitir	Nginx encaminha /api/* ao backend
R3	sub-app	sub-dados (PostgreSQL)	TCP 5432	Permitir	Backend acessa o banco
R4	Internet	sub-app e sub-dados	Qualquer	Negar	Nada da internet chega direto à aplicação ou ao banco
R5	sub-publica	sub-dados	Qualquer	Negar	O gateway não fala com o banco
R6	sub-dados	Internet	Qualquer	Negar	Banco sem saída para a internet

Padrão: tudo que não está listado é negado. As regras são *stateful* (a resposta de uma conexão permitida volta automaticamente).

Regra de ouro atendida: o banco está na sub-rede privada de dados, só é acessível a partir da sub-rede de aplicação na porta 5432 e não tem entrada nem saída para a internet.

Sobre o Supabase: é um PostgreSQL gerenciado. No ambiente projetado ele é tratado como a sub-rede de dados, sem endereço público e acessível apenas pela sub-rede de aplicação por conexão privada. No desenvolvimento atual o backend conecta ao pooler do Supabase pela internet com TLS, o que é aceitável só para o MVP.

7 Legenda e convenções

As mesmas convenções estão desenhadas na caixa de legenda do próprio diagrama.

Cores — onde cada um mora

Cor	Significado
	Sub-rede pública (recebe tráfego da internet) — Nginx
	Sub-rede privada de aplicação — Backend API
	Sub-rede privada de dados (sem internet) — PostgreSQL
	Internet / serviços de terceiros (fora da VPC) — navegador, OSRM, OpenStreetMap
	Borda grossa escura: limite da VPC (rede principal 10.0.0.0/16)
	Amarelo: dado guardado em memória dentro do processo do backend

Linhas — como cada um fala

Linha	Significado
	Contínua preta: chamada síncrona (requisição/resposta) ativa hoje
	Contínua laranja: chamada síncrona planejada — já configurada, mas ainda sem tráfego real
(não usada)	Tracejada = comunicação assíncrona (fila/evento). Não aparece porque o projeto não tem fila.
(não usada)	Pontilhada = observabilidade (métricas/logs). Não aparece porque não há coleta implementada.
[1] HTTPS / 443	Rótulo de toda seta: [nº] PROTOCOLO / porta . O número liga a seta à linha da tabela da seção 5.

Formas — quem é quem

Forma	Significado
	Usuário (navegador)
	Container / serviço que executa código (Nginx, Backend API)
	Armazenamento de dados: banco (contorno roxo) ou estado em memória (amarelo, desenhado dentro do backend porque faz parte do mesmo processo)
	Serviço de terceiro na internet (OSRM, OpenStreetMap)
Retângulo grande	Rede ou sub-rede; o componente desenhado dentro dele pertence àquela rede

Verificação cruzada

- ✓ **Todo endpoint da tabela está exposto por algum componente do diagrama:** todos ficam no Backend API (base /api/v1), que mostra seus controllers e os recursos planejados; o Nginx apenas repassa /api/*.
- ✓ **Toda seta do diagrama tem uma linha na tabela de protocolos:** setas [1] a [5] ↔ linhas 1 a 5 da seção 5, com o mesmo protocolo e porta.
- ✓ **Todo componente do diagrama está em alguma sub-rede:** Nginx → sub-pública; Backend API e estado em memória → sub-app; PostgreSQL → sub-dados. Navegador, OSRM e OpenStreetMap ficam, de propósito, na Internet (fora da VPC).
- ✓ **Inventário ↔ diagrama:** os 7 componentes do inventário (3.1) estão no diagrama, e nenhum componente foi inventado.